

JAVA Programming (CST015)

Lecture 6: Iteration Statements in JAVA



Department of Computer Science and Engineering
National Institute of Technology, Srinagar, Jammu and Kashmir
March 10, 2026

Iteration Statements or Loops

- Iteration Statements are also called as Looping Statements.
- Loops can execute a block of code as long as a specified condition is reached and as long as termination condition is not met.
- Loops are handy because they save time, reduce errors, and they make code more readable.
- Java's iteration statements are **for, while, and do-while**

The While Statements

The while loop is Java's most fundamental loop statement.

The while loop loops through a block of code as long as a specified condition is true.

A while statement has the following syntax:

```
while (condition)  
    statement;
```

- The condition can be any Boolean Expression.
- If the condition is true, the statement is executed; then the condition is evaluated again
- The statement is executed over and over until the condition becomes false
- If the condition of a while statement is false initially, the statement is never executed.
- When condition becomes false, control passes to the next line of code immediately following the loop
- Therefore, we say that a while statement **executes zero or more times**

Example

```
class demo{  
    public static void main(String[] args)  
{  
    int a=15;  
    while(a>10)  
    {  
        System.out.println("Value of "+a+" + is greater than 10");  
        a--;  
    }  
}  
}
```

While with no Body

```
class demo{  
    public static void main(String[] args)  
{  
    int a=15;  
    while(a>10);  
}  
}
```

The body of the while (or any other of Java's loops) can be empty.

This is because a null statement (one that consists only of a semicolon) is syntactically valid in Java

Output?

```
class demo{  
    public static void main(String[] args)  
{  
    int a=15;  
    while(a>10);  
    System.out.println("hello");  
}  
}
```

Do-While Loop

- The do-while loop is a variant of the while loop.
- if the conditional expression controlling a while loop is initially false, then the body of the loop will not be executed at all.
- However, sometimes it is desirable to execute the body of a loop at least once, even if the conditional expression is false to begin with.
- This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

```
do {  
    // code block to be executed  
}  
while (condition);
```

The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested.

Example

```
class demo{  
    public static void main(String[] args)  
{  
    int a=15;  
    do  
    {  
        System.out.println("Value of "+a+" is greater than 10");  
        a--;  
    } while(a>10);  
}  
}
```


Do-While Loop

- The do-while loop is especially useful when you process a menu selection, because you will usually want the body of a menu loop to execute at least once.

```
char choice;

do {
    System.out.println("Help on:");
    System.out.println("  1. if");
    System.out.println("  2. switch");
    System.out.println("  3. while");
    System.out.println("  4. do-while");
    System.out.println("  5. for\n");
    System.out.println("Choose one:");
    choice = (char) System.in.read();
} while( choice < '1' || choice > '5');
```

For Loop

```
for (initialization; condition; increment)  
statement;
```

- This is equivalent to

```
initialization;  
while (condition) {  
    statement;  
    increment;  
}
```

JAVA For Loop

- The **initialization** is always performed **once**
- The condition of a ***for*** statement is tested *prior* to executing the loop body (like in the *while* statement)
- Therefore, a ***for*** loop will execute **zero or more times**
- For loops are well suited for cases where the number of iterations is known beforehand
- The increment/decrement is executed after each iteration of the loop

The `for` Statement: examples

```
for (int count=1; count < 75; count++) {  
    System.out.println(count);  
}
```

```
for (int num=1; num <= max; num = num * 2) {  
    System.out.println ("Next power of 2: " + num);  
}
```

Omitting parts in a *for* Statement

- Each **expression** in the header of a **for** loop is **optional**
 - If the **initialization** is **left** out, no initialization is performed
 - If the condition is left out, it is always considered to be true, and therefore makes an infinite loop
 - If the increment is left out, no increment operation is performed. This will also make the loop infinite.
- Both semi-colons are always required

```
for (;;) { // an infinite loop
    System.out.println ("beep");
}
```

Output?

```
public class Main {  
  
    public static void main(String[] args) {  
        for(int i=0;i<10;i++)  
        {  
        }  
    }  
}
```

Output?

```
public class Main {  
    public static void main(String[] args) {  
        for(int i=0;i<10;i++)  
        {  
            //System.out.println();  
        }  
        System.out.println(i);  
    }  
}
```

Output?

```
public class Main {  
    static int i=0;  
    public static void main(String[] args) {  
        for(;i<10;i++)  
        {  
            System.out.println(i);  
        }  
    }  
}
```


Output?

```
class demo{
    static int i=20;
    public static void main(String[] args)
    {

        for (int i=5;i<10;i++)
        {
            System.out.println(i);
        }

        System.out.println(i);
    }
}
```

Output?

```
public class Main {  
    static int i=0;  
    public static void main(String[] args) {  
        for(;;i++)  
        {  
            System.out.println(i);  
        }  
    }  
}
```

Output?

```
public class Main {  
    static int i=0;  
    public static void main(String[] args) {  
        for(;i<10;)   
        {  
            System.out.println(i);  
        }  
    }  
}
```

Exercise: WAP a Java Program to print the Multiplication table

```
enter the number whose table you want to display
```

```
10
```

```
10x1 = 10
```

```
10x2 = 20
```

```
10x3 = 30
```

```
10x4 = 40
```

```
10x5 = 50
```

```
10x6 = 60
```

```
10x7 = 70
```

```
10x8 = 80
```

```
10x9 = 90
```

```
10x10 = 100
```

Solution:

```
class demo{  
  
    public static void main(String[] args)  
    {  
  
        int num=10;  
        for(int i=1;i<=10;i++)  
        {  
            int mul=i*num;  
            System.out.println(num+" *"+i+" = "+mul);  
        }  
    }  
}
```

Exercise: WAP to find the factorial of a number using Recursion

Factorial Program:

```
public class demo{  
    static int fact(int num)  
    {  
        int factorial=1;  
        if (num==1)  
            return factorial;  
        else  
            return factorial=num*fact(num-1);  
    }  
  
    public static void main(String[] args)  
    {  
        int num=5;  
  
        System.out.println(fact(5));  
    }  
}
```

Jump Statements

Java supports three jump statements:

1. **break:**
2. **continue**
3. **return.**

Break

break can be used to exit out of a loop.

Break can also be used like a **goto**.

```
class BreakLoop {  
    public static void main(String args[]) {  
        for(int i=0; i<100; i++) {  
            if(i == 10) break; // terminate loop if i is 10  
            System.out.println("i: " + i);  
        }  
        System.out.println("Loop complete.");  
    }  
}
```

Labelled Break: Using break as a Form of Goto

- Break can also be used like a goto statement.
- Java does not have a goto statement.
- In case of nested loops, the break brings the control out of the inner loop only.
- So, in such scenarios where we want the break to bring the control out from the outer loops, labelled break can be used.
- The general form of the labeled break statement is shown here:

`break label;`

label is the name of a loop that identifies a block of code.

A **label** is simply a name given to a loop.

When we use `break labelName;`, the program **immediately terminates the loop that has that label**, even if the break is inside a deeper nested loop.

What will be the output of this program?

```
public class demo2 {  
    public static void main(String[] args)  
    {  
        for(int i=0; i<5;i++)  
        {  
            for(int j=0;j<5;j++)  
            {  
                System.out.println("inner loop"+j);  
            }  
            System.out.println("outer loop"+i);  
        }  
    }  
}
```

Break used here is bringing the control out of the inner loop only.

```
public class demo2 {  
    public static void main(String[] args)  
    {  
        for(int i=0; i<5;i++)  
        {  
            for(int j=0;j<5;j++)  
            { if (j==2)  
                {  
                    break;  
                }  
            System.out.println("inner loop"+j);  
            }  
            System.out.println("outer loop"+i);  
        }  
    }  
}
```

Labelled Break

Labelled Break will bring the control out of the outer loops also.

```
public class demo2 {  
    public static void main(String[] args)  
    {  
        label1:  
        for(int i=0; i<5;i++)  
        {  
            for(int j=0;j<5;j++)  
            { if (j==2)  
                {  
                    break label1;  
                }  
            System.out.println("inner loop"+j);  
            }  
            System.out.println("outer loop"+i);  
        }  
        System.out.println("hi");  
    }  
}
```

Output?

```
class demo{  
  
    public static void main(String[] args)  
    {  
  
        label1:  
        System.out.println("hello");  
        break label1;  
    }  
}
```

Answer to Previous QuestionOutput?

```
class demo{  
  
    public static void main(String[] args)  
    {  
  
        label1:  
        System.out.println("hello");  
        break label1;  
    }  
}
```

Error: Break can only be used inside loop and switch.

Continue Statement

Continue causes the current iteration to skip and move to the next iteration.

```
public class demo3 {  
    public static void main(String[] args)  
    {  
        for(int i=0;i<10;i++)  
        {  
            if (i==2)  
            {  
                continue;  
            }  
            System.out.println(i);  
        }  
    }  
}
```


return Statement

- The return statement is used to **explicitly return** from a **method**.
- That is, it causes program control to **transfer back** to the **caller of the method**.
- At any time in a method, the return statement can be used to cause execution to branch back to the caller of the method.

```
public class demo5 {  
    static void fun(int a)  
    {  
        System.out.println("hello1");  
        if (a==2)  
            return;  
        System.out.println("This will not execute");  
    }  
    public static void main(String[] args) {  
        fun(a: 2);  
    }  
}
```

Output?

```
public class demo{  
  
    public static void main(String[] args)  
{  
  
    int num=5;  
    System.out.println("hello");  
    return;  
    System.out.println("bye");  
}  
}
```

answer to previous question

```
public class demo{  
  
    public static void main(String[] args)  
{  
    int num=5;  
    System.out.println("hello");  
    return;  
    System.out.println("hello");  
}  
}
```

error: unreachable statement

```
    System.out.println("hello");
```

Output?

```
public class demo{  
  
    public static void main(String[] args)  
    {  
  
        int num=5;  
        if (num==5)  
            return;  
        System.out.println("hello");  
    }  
}
```

Answer to previous Code?

```
public class demo{  
  
    public static void main(String[] args)  
{  
  
    int num=5;  
    if (num==5)  
        return;  
    System.out.println("hello");  
}  
}
```

This will compile with no errors as now the program will execute the return statement only if value of num is 5.

Output?

```
class demo{
    public static void main(String[] args)
    {
        for (int i = 0; i < 5; i++)
        {
            if (i == 2)
            {
                continue;
            }
            System.out.println(i + " ");
        }
    }
}
```

Solution to Previous Question

```
class demo{
    public static void main(String[] args)
    {
        for (int i = 0; i < 5; i++)
        {
            if (i == 2)
            {
                continue;
            }
            System.out.println(i + " ");
        }
    }
}
```

Output:

0

1

3

4